SceneFork — Master Technical Blueprint

Upload any 10-second video clip. Speak new rules of physics. Play inside your video in the browser.

Meta Global AI Developer Hackathon

SAM 3.1 + Muse Wonder + Muse Image + Muse Voice + Muse Glimmer + Muse Code

Target: Grand Prize & $20k Most Viral Bonus

1. Strategic Alignment with Hackathon Judging Rubric (100%)

Criterion	Weight	SceneFork Winning Strategy
Real-World Impact	**30%**	Turns passive video into interactive 2.5D physics simulations, playable game prototypes, and interactive educational labs in under 10 seconds—eliminating hours of manual rotoscoping in After Effects, clean-plating in Photoshop, and rigid-body rigging in Unity.
Meta AI Tooling Depth	**25%**	Orchestrates **6 distinct Meta models** where each model's output feeds the next: `SAM 3.1` masks feed `Muse Image` clean-plating and `Muse Wonder` physics inference; `Muse Voice Transcribe` + open-weight `Muse Glimmer` (on Modal/Baseten) route deictic cursor commands into `Muse Code`.
Technical Depth & Agentic Loop	**20%**	Includes OpenCV Douglas-Peucker polygon hull simplification for 60 FPS WASM physics, 15px morphological shadow-halo dilation, and a **30-tick headless WebWorker physics verifier** that tests and self-heals `Muse Code` patches before live injection.
Live Demo & Virality	**25%**	High visual drama: pause a smartphone street video, grab a parked delivery van right out of the scene with your cursor, fling it into a streetlamp with procedural metallic crash audio, and speak new laws of gravity live on stage.

2. System Architecture & The "Timeline Fork" Mechanism

> **Core Engineering Insight:** Once real-time physics or user interaction takes over at frame t_{fork}, objects leave their pre-recorded video paths. SceneFork bridges linear video and interactive simulation via a three-stage **Freeze, Clean-Plate, and Fork** transition.

1. **Linear Playback State ($t = 0 \rightarrow t_{fork}$):** The user watches or scrubs a standard 1080p MP4 clip in the browser and clicks **Fork Scene** (or speaks a trigger phrase) at any frame t_{fork}.
2. **Parallel Multimodal Deconstruction (at t_{fork}):**
   - **Spatial & Temporal Cutouts (`SAM 3.1` on Modal GPU):** Segments every distinct object at t_{fork}, extracts RGBA sprites, computes initial optical-flow velocities from t_{fork} - *0.15s*, and runs Douglas-Peucker polygon hull simplification.

- **Dilated Background Inpainting (`Muse Image`):** Takes the union of all dynamic `SAM 3.1` masks dilated by 15 pixels (to erase drop-shadows and compression fringes) and inpaints a clean background plate.
- **Physical Property Grounding (`Muse Wonder`):** Inspects the numbered `SAM 3.1` mask contours and assigns `body_type` (`fixed` vs. `dynamic`), `mass_kg`, `restitution`, `friction`, `acoustic_material`, and optional hinge joints.

3. **60 FPS Interactive Takeover ($t > t_{fork}$):** The HTML5 video crossfades seamlessly into a **PixiJS v8 + Rapier2D WebAssembly** canvas where every segmented object is a live rigid body over the clean-plated background.

3. Six-Model Orchestration Specification

Pipeline Node	Meta Model	Input Payload	Output Artifact	Latency Target
1. Hull & Sprite Extractor	`SAM 3.1`	Keyframe t_{fork} RGB image + auto grid/point prompts	Per-object RGBA sprites + 8-16 vertex convex hull polygons	~420 ms (Modal H100)
2. Background Clean-Plate	`Muse Image`	Keyframe t_{fork} + 15px dilated union foreground mask	Unoccluded background plate + optional stylized sprite skins	~1,100 ms (Async parallel)
3. Physics Grounding	`Muse Wonder`	Annotated frame with numbered mask contours + centroids	Strict `ScenePhysicsManifest` JSON (mass, bounce, material)	~850 ms (Async parallel)
4. Voice Stream	`Muse Voice Transcribe`	WebRTC / WebSocket microphone PCM stream + word timestamps	Time-aligned transcript with deictic trigger timestamps	Streaming (<150 ms)
5. Fast Deictic Router	`Muse Glimmer` / `Spark Open` (Baseten)	Transcript + cursor `(x,y)` hover history + active mask IDs	Resolved entity bindings (*"this truck"* → *ent_mask_02*) + fast param tweaks	~85 ms (Baseten vLLM)
6. World Rule Synthesizer	`Muse Code`	Resolved intent + `SceneForkMutator` TypeScript contract	Hot-swappable JS/TS ECS plugin verified in 30-tick shadow world	~1,600 ms

4. Production Data Contracts & Prompts

4.1 `ScenePhysicsManifest` JSON Contract

```
{
  "scene_id": "urban_street_demo",
  "fork_timestamp_sec": 3.42,
  "resolution": { "width": 1280, "height": 720 },
  "clean_plate_url": "/assets/urban_street_cleanplate.png",
  "world_config": { "gravity": { "x": 0.0, "y": 980.0 }, "pixels_per_meter": 100.0 },
  "entities": [
    {
      "id": "ent_mask_01",
      "mask_id": 1,
      "label": "street_asphalt_ground",
      "body_type": "fixed",
      "centroid_px": { "x": 640, "y": 650 },
      "hull_vertices_local_px": [[-640, -40], [640, -40], [640, 70], [-640, 70]],
      "sprite_url": "/assets/mask_01_rgba.png",
      "physics": { "mass_kg": 0.0, "restitution": 0.3, "friction": 0.9 },
      "acoustic_material": "concrete",
      "z_index": 1
    },
    {
      "id": "ent_mask_02",
      "mask_id": 2,
      "label": "red_food_truck",
      "body_type": "dynamic",
      "centroid_px": { "x": 420, "y": 490 },
      "hull_vertices_local_px": [[-140, -85], [135, -85], [145, 80], [-145, 80]],
      "sprite_url": "/assets/mask_02_rgba.png",
      "physics": { "mass_kg": 1200.0, "restitution": 0.2, "friction": 0.7 },
      "acoustic_material": "hollow_metal",
      "z_index": 3
    }
  ]
}
```

4.2 System Prompts for Muse Wonder & Muse Code

```
// 1. MUSE WONDER SYSTEM PROMPT (Physical Grounding)
You are the Physical World Grounding Engine for SceneFork.
Given a video frame with numbered SAM 3.1 segmentation contours [Mask #ID]:
1. Classify structural surfaces (floors, roads, walls, tables) as `body_type: "fixed"`.
2. Classify movable objects (vehicles, cups, balls, people, props) as `body_type: "dynamic"`.
3. Infer realistic `mass_kg` (0.1 to 5000), `restitution` (0.05 to 0.90), and `friction` (0.1 to
0.95).
4. Tag `acoustic_material` from: ["hollow_metal", "glass", "ceramic", "wood", "rubber", "concrete"].
Output strictly valid JSON conforming to ScenePhysicsManifest.

// 2. MUSE CODE SYSTEM PROMPT (Sandboxed ECS Mutator Synthesis)
You are the SceneFork Real-Time World Mutator Agent powered by Muse Code.
Generate a single JavaScript object implementing the SceneForkMutator interface:
({
  id: "mutator_slingshot_truck",
  description: "Turns ent_mask_02 into an interactive slingshot and makes ent_mask_01 bouncy",
  onInit(ctx) {
    ctx.getEntity("ent_mask_01")?.setRestitution(0.92);
  },
  onTick(ctx, dt) {},
  onPointerDragRelease(ctx, entityId, dragVec) {
    if (entityId === "ent_mask_02") {
      ctx.getEntity(entityId)?.applyImpulse(-dragVec.x * 35, -dragVec.y * 35);
      ctx.audio.playNote(220, 0.3, "triangle");
    }
  },
  onCollision(ctx, idA, idB, impulse) {
    if (impulse > 120) ctx.spawnParticleBurst(idA, 25, { style: "confetti" });
  }
})
Safety rules: Never apply impulses > 50000; guard all divisions with Math.max(dist, 10).
```

5. Core Engineering Modules (Backend & Frontend)

5.1 OpenCV Hull Simplifier & 15px Shadow-Halo Dilator (Python)

```python
def compile_sam31_mask_to_hull(frame_bgr, binary_mask, epsilon_factor=0.014, dilation_px=15):
    contours, _ = cv2.findContours(binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    largest = max(contours, key=cv2.contourArea)
    M = cv2.moments(largest)
    cx, cy = int(M["m10"] / max(M["m00"], 1e-5)), int(M["m01"] / max(M["m00"], 1e-5))

    # Convex hull + Douglas-Peucker simplification (8-16 vertices for 60 FPS Rapier2D)
    hull = cv2.convexHull(largest)
    approx = cv2.approxPolyDP(hull, epsilon_factor * cv2.arcLength(hull, True), True)
    local_vertices = [[int(pt[0][0] - cx), int(pt[0][1] - cy)] for pt in approx]

    # 15px dilation ensures drop-shadows and compression halos are erased by Muse Image
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (dilation_px * 2 + 1, dilation_px * 2 + 1))
    dilated_mask = cv2.dilate(binary_mask, kernel, iterations=1)
    return cx, cy, local_vertices, dilated_mask
```

5.2 Self-Healing 30-Tick Headless Physics Critic (TypeScript)

> **Why Judges Love This (20% Technical Depth Rubric):** Instead of blindly running LLM-generated code on the main canvas, SceneFork tests every Muse Code plugin for 30 frames inside an isolated off-screen Rapier2D world. If any entity velocity explodes or returns NaN, the error telemetry is fed back to Muse Code to self-repair before hot-swapping.

```
export function verifyMutatorHeadless(codeStr: string, entities: Snapshot[]): ValidationResult {
  const mutator = new Function(`return (${codeStr});`)();
  const shadowWorld = new RAPIER.World({ x: 0.0, y: 980.0 });
  // Instantiate shadow bodies, run mutator.onInit(mockCtx), and step 30 ticks (0.5s simulated time)
  for (let tick = 0; tick < 30; tick++) {
    mutator.onTick?.(mockCtx, 1 / 60);
    shadowWorld.step();
    for (const [id, body] of shadowBodies) {
      const v = body.linvel();
      if (!Number.isFinite(v.x) || Math.hypot(v.x, v.y) > 15000) {
        shadowWorld.free();
        return { valid: false, reason: `Velocity explosion on ${id} at tick ${tick}` };
      }
    }
  }
  shadowWorld.free();
  return { valid: true };
}
```

6. 48-Hour Hackathon Build Schedule & Team Split

Hours	Frontend & Physics Lead (PixiJS + Rapier2D)	AI & Vision Backend Lead (Modal + Meta APIs)
00 – 08	Scaffold Vite + React + PixiJS v8 + `@dimforge/rapier2d-compat`. Render static clean-plate + draggable/throwable polygon sprites at 60 FPS.	Deploy `SAM 3.1` on Modal GPU. Build OpenCV Douglas-Peucker hull simplifier and 15px dilated mask generator.
08 – 18	Build the **X-Ray Debug HUD** (neon polygon outlines, centroid markers, velocity vectors) and `ProceduralAudioEngine` WebAudio synth.	Wire `Muse Image` background clean-plating and `Muse Wonder` `ScenePhysicsManifest` JSON inference in parallel via `asyncio.gather`.
18 – 30	Implement `DeicticTracker.ts` (cursor hover history during speech) and the `HeadlessVerifier.ts` 30-tick shadow world critic.	Integrate `Muse Voice Transcribe`, `Muse Glimmer` (on Baseten for <100ms intent routing), and `Muse Code` ECS plugin synthesis loop.
30 – 40	Build the seamless `VideoForkTimeline.tsx` crossfader and live Agent Telemetry drawer showing `Muse Code` self-healing in real time.	Add `Muse Image` mask-constrained sprite restyling (e.g., *"reskin the street into comic book ink / stained glass"*).
40 – 48	Pre-bake fallback caches for the 3 Golden Demo Clips, record the 3-minute viral pitch video, and rehearse live stage transitions.	Set up keep-alive pings on Modal/Baseten endpoints so zero GPU cold starts occur during live judging.

7. Pre-Planned "3 Golden Clips" & Risk Mitigation Checklist

- **Golden Clip 1 — Urban Street Skate & Food Truck (Physics & Slingshot Hook):** Fork mid-jump, grab the parked food truck out of the frame revealing the clean-plated street behind it, bounce the skateboarder off the awning, and command via voice: *"Turn this truck into an Angry Birds slingshot and make the cones explode into confetti on impact."*
- **Golden Clip 2 — Tabletop Mugs & Glassware (Procedural Audio & Style Hook):** Fork a clip of 5 coffee mugs on a desk. Command via voice: *"Turn these mugs into a playable lo-fi marimba, spawn bouncing rain droplets from the top, and reskin the mugs into stained glass."*
- **Golden Clip 3 — Live Stage Webcam Capture (The Judge Mic-Drop):** Record a 4-second webcam clip of your own desk with a real bottle and notebook on stage, click **Fork Scene**, and immediately toss your physical desk objects around inside the browser window.
- **Stage Network Insurance:** Store content-addressed SHA-256 caches of the `SAM 3.1` + `Muse Image` outputs for Golden Clips 1 & 2 locally so even if venue Wi-Fi drops, the live voice + `Muse Code` + physics engine runs 100% smoothly.